tethys_example

May 22, 2025

1 Getting started with Tethys

1.1 Motivation

Integrated human-Earth systems models, such as GCAM, can project future water demand at a coarse, regionally-relevant scale by modeling long-term interactions between multiple sectors under a variety of scenarios, while gridded hydrology models simulate physical processes at a much finer spatial and temporal resolution. **tethys** facilitates coupling between these kinds of models by providing finer-scale water demand data while maintaining consistency with coarser-scale global dynamics.

While **tethys** is designed to integrate seamlessly with GCAM, it has the ability to downscale region-scale water demand data from other sources as well.

1.2 Installation

As a prerequisite, you'll need to have Python installed (version 3.9 or above), and if you plan on querying a GCAM database, Java must be installed and added to your path.

tethys can be install from PyPi via pip:

```
pip install tethys-downscaling
```

This will automatically install the dependencies. In order to avoid package version conflicts, consider using a virtual environment.

Try importing **tethys** to confirm that installation was successful:

[1]:
```
import tethys

tethys.__version__  # should print the version number
```

[1]:
```
'2.1.0'
```

[2]:
```
# load additional packages needed for this tutorial
# all are included with the tethys install
from pathlib import Path
from dask.distributed import Client
from matplotlib import colors
from matplotlib import pyplot as plt
import os
```

1.3 Example Data

Example data and configurations can be downloaded from Zenodo here, or by using the following:

`tethys.get_example_data()`

The download decompresses to about 4.5GB. By default, it will make a directory called `example` at the root of the tethys pacakge, but you can specify another path.

Some systems do not do well unpacking data that size without first having the zipped archived downloaded. If you would rather have the code download the zipped archive first, then unpack it, then clean up you can run:

`tethys.get_example_data(download_first=True)`

[3]:
```
os.mkdir("example_data")
tethys.get_example_data(download_first=True,␣
 ↪example_data_directory="example_data")
```

```
Downloading example data for Tethys version 2.1.0.

100%|      | 1.37G/1.37G [08:59<00:00, 2.54MiB/s]

Downloaded example_data/example.zip
Extracted example_data/example.zip to example_data and deleted the zipped file
```

1.4 Running tethys

With the example data downloaded, a simple configuration can be run by the following:

[4]:
```
# assuming you downloaded to the default location
config_file = "example_data" + '/example/config_example.yml'
```

Let's take a look at the config file:

[5]:
```
with open(config_file) as cfg:
    print(cfg.read())
```

```
# Basic example config file for Tethys

# Project level settings
years: [2010, 2015, 2020, 2025, 2030]
resolution: 0.125
demand_type: withdrawals

# output settings
output_file: output/example_output.nc

# csv input
csv: data/example_data.csv

# dictionary of proxy files and promised variables and years
```

```
proxy_files:
  data/population/ssp2_{year}.tif:
    variables: Population
    years: [2010, 2020, 2030]

# mapping of inputs sectors to proxies
downscaling_rules:
  Municipal: Population

# list of map files
map_files:
  - data/maps/regions.tif
```

[6]:
```
# run tethys using the config file
result = tethys.run_model(config_file)
```

```
Loading Proxy Data
Interpolating Proxies
Downscaling Municipal
```

[7]:
```
# note that the configuration opions used to conduct the run
# will be present in the `result` variable

result.years
```

[7]:
```
[2010, 2015, 2020, 2025, 2030]
```

[8]:
```
result.csv
```

[8]:
```
'data/example_data.csv'
```

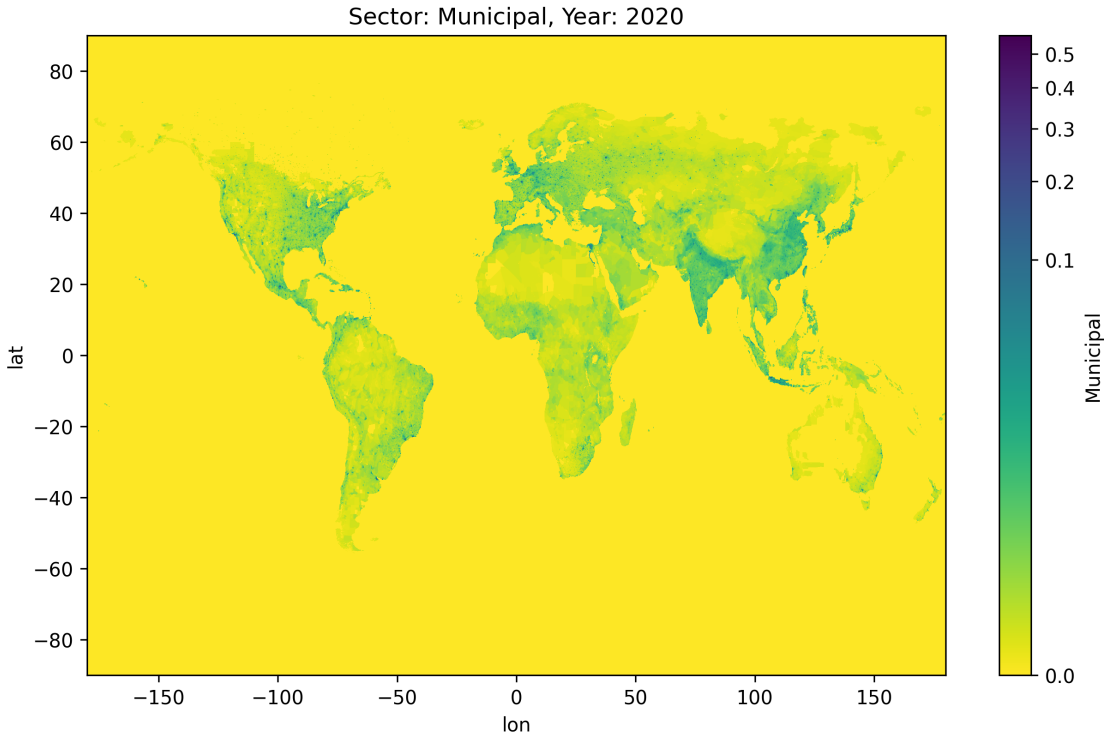
1.5 Plotting

tethys makes use of the Xarray package, which provides convenient plotting functionality.

View the output of Tethys for the Municipal sector for year 2020

[9]:
```
# higher dpi in order to see resolution
plt.figure(figsize=(10, 6), dpi=300)

# powernorm the color palette in order to see more detail at the low end
result.outputs.Municipal.sel(year=2020).plot(
    norm=colors.PowerNorm(0.25),
    cmap='viridis_r',
)

plt.title("Sector: Municipal, Year: 2020")
```

```
plt.show()
```



Interactively view the input data before downscaling for the Municipal sector for year 2020

[10]:
```
from tethys import InputViewer

# instantiate input viewer
gridded_inputs = InputViewer(
    result,
    sector="Municipal",
    year=2020,
)
```

The data can now be plotted...

[11]:
```
gridded_inputs.plot(cnorm="log")
```

[11]:
```
:Image   [x,y]   (inputs)
```

Plotting options can be viewed by calling `help` on the function:

[12]:
```
help(gridded_inputs.plot)
```

```
Help on method plot in module tethys.utils.spatial:

plot(width: int = 1000, height: int = 600, cnorm: Optional[str] = None, cmap:
str = 'viridis_r') method of tethys.utils.spatial.InputViewer instance
    Generate an interactive plot.

    :param width: Width of the plot in pixels, defaults to 1000.
    :type width: int, optional
    :param height: Height of the plot in pixels, defaults to 600.
    :type height: int, optional
    :param cnorm: Normalization for the colormap, can be a string or None,
defaults to None.
    :type cnorm: Union[str, None], optional
    :param cmap: Colormap to use for the plot, defaults to "viridis_r".
    :type cmap: str, optional
    :return: An interactive plot object.
    :rtype: hvPlot
```

The underlying data can also be exported to a raster file using the `.to_raster()` method:

```
gridded_inputs.to_raster("<your raster path>")
```

1.6 Dashboard

tethys uses Dask for parallelization and to lazily compute results. You can launch the dask distributed client in order to view dashboard and monitor the progress of large workflows.

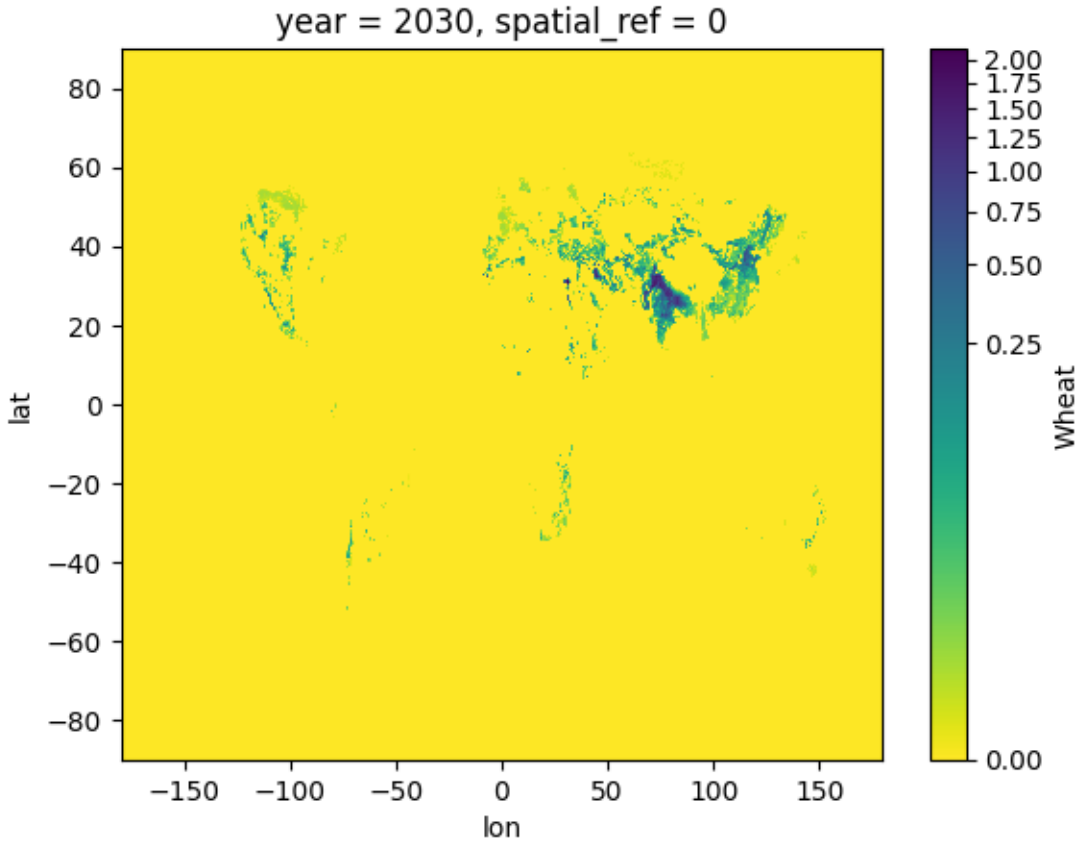
NOTE: Viewing the dashboard requires a few other dependencies not automatically installed by tethys. Please ensure your local Java library has been added to your path.

[13]:
```
# this configuration may need to be different depending on your machine
client = Client(
    threads_per_worker=8,
    n_workers=1,
    processes=False,
    memory_limit='8GB',
    dashboard_address=":46321"
)

# link to view the dask dashboard in your browser, probably localhost:8787
print(f"Click link to open dashboard: {client.dashboard_link}")

# run tethys AFTER launching the client
config_file = 'example_data' + '/example/config_demeter.yml'
result = tethys.run_model(config_file)

# this configuration does not write outputs to a file,
# so plots are lazily computed when requested
```

```
result.outputs.Wheat.sel(year=2030).plot(
    norm=colors.PowerNorm(0.25),
    cmap='viridis_r'
)

plt.show()
```

```
Click link to open dashboard: http://172.17.0.2:46321/status
Loading Proxy Data
Interpolating Proxies
Database scenarios: Reference
Downscaling Irrigation
```

```
/opt/conda/lib/python3.11/site-packages/dask/array/core.py:4988:
PerformanceWarning: Increasing number of chunks by factor of 48
  result = blockwise(
```



1.7 Additional

Check out the full user guid on our documentation website!